Socket ZH

Összesen 20 pont (6 + 6 + 8). A 2. és 3. feladatban a program a `localhost` címen figyeljen. A tesztszerverek is a `localhost` címen érhetők el. A csomag: `zh_A.zip`.

1. feladat: járatfoglalás (6 pont)

1. feladat

Készítsünk egy TCP klienst! A megoldás helye: `feladat1/client.py`, indítása: `python client.py localhost 20001`

A kliens küldje el az Ön neptun kódját, a járatszámot és az üzenet típusát (`"request"`) egy üzenetben a tesztszervernek, ahol a formátum: `(6s 8s 7s)`
A keresett járatszám: `LH2048`.

A szerver válaszüzenete a szabad helyek száma és az utazási osztály, formátum: `(i 10s)`
Az utazási osztály értékei lehetnek: `"economy"`, `"business"`.

Önnek legalább 3 szabad helyre van szüksége economy osztályon.

Ha a visszakapott osztály nem economy, vagy a szabad helyek száma kisebb mint 3, akkor a kliens válaszüzenete: (neptun kód, járatszám, `"reject"`), formátum: `(6s 8s 7s)`

Ha a feltételek teljesülnek, akkor a kliens válaszüzenete: (neptun kód, járatszám, `"reserve"`), formátum: `(6s 8s 7s)`

A szerver végső válasza (`"reject"` és `"reserve"` esetén is) egy foglalási azonosító, formátum: `(12s)`
Ha hiba van a megoldásában, a válaszüzenetben egy-két esetben jelezve van, hogy mi a probléma; a részleteket a tesztszerver naplója mutatja.

Ellenőrzés. Töltse le és csomagolja ki a zárthelyi csomagját (zip), a feladatot a kicsomagolt mappában oldja meg! A Docker Desktopnak futnia kell. A csomag mappájában (ahol a `tests.py` van):

```
python tests.py check t8gNJ6rNm3ST feladat1
```

A következő kód (`t8gNJ6rNm3ST`) ennek a feladatnak a saját kódja: pontosan másolja ki (a kis- és nagybetű számít)! A `check` elindítja a programját, lefuttatja a teszteket, és a végén kiírja a kódokat: ezeket őrizze meg. Kézi mód fejlesztéshez: `python tests.py start t8gNJ6rNm3ST feladat1` (ez az ablak a tesztek naplója), és egy másik ablakban a saját programja: `python feladat1/client.py localhost 20001`.

Fontos: a `tesztek` mappa obfuszkált kódját „fekete dobozként” kell kezelni, a rá alkalmazott kódvisszafejtés („reverse engineering”) csalásnak minősül.

2. feladat: raktárkészlet (6 pont)

2. feladat

Készítsünk egy TCP szervert, amely a `select` segítségével egyszerre több kliens kiszolgálására is képes! A megoldás helye: `feladat2/server.py`, indítása: `python server.py <port>`

A szerver különböző raktárak készletadatait tárolja. A kliensek háromféle parancsot küldhetnek a szervernek:

- `ADD W P Q` – adjon hozzá Q darabot a W raktárban a P termékhez
- `GET W P Q` – kérdezze le a W raktárban a P termék aktuális darabszámát (a Q nem releváns, 0-ra van állítva)
- `TOT W P Q` – számítsa ki a W raktár teljes készletét, azaz a benne lévő termékek összdarabszámát (a P, Q nem releváns, P `"AAAAAAAA"`-ra, Q 0-ra van állítva)

A parancsokat a kliensek struct formátumban küldik: `(3s 6s 8s i)`
A szerver minden parancs után egy egész számot ad vissza `(i)`, amely ADD és GET után a termék aktuális darabszáma (az adott raktárban), TOT után a raktár teljes készlete.

Példa:

```
"ADD", "WH001", "Laptop", 10    → a szerver hozzáad 10 darab Laptop terméket a WH001 raktárhoz és visszaküldi a 10-et
"GET", "WH001", "Laptop", 0     → a szerver visszaküldi a Laptop aktuális darabszámát, a 10-et
"ADD", "WH001", "Mouse", 5      → a szerver hozzáad 5 darab Mouse terméket és visszaküldi az 5-öt
"TOT", "WH001", "AAAAAAAA", 0   → a szerver visszaküldi a WH001 raktár teljes készletét, a 15-öt
```

A kliens nem fog szándékosan helytelen üzenetet (pl. érvénytelen parancsot stb.) küldeni.

Ellenőrzés. Töltse le és csomagolja ki a zárthelyi csomagját (zip), a feladatot a kicsomagolt mappában oldja meg! A Docker Desktopnak futnia kell. A csomag mappájában (ahol a `tests.py` van):

```
python tests.py check k7m2x9p4qw8r feladat2
```

A következő kód (`k7m2x9p4qw8r`) ennek a feladatnak a saját kódja: pontosan másolja ki (a kis- és nagybetű számít)! A `check` elindítja a programját, lefuttatja a teszteket, és a végén kiírja a kódokat: ezeket őrizze meg. Kézi mód fejlesztéshez: `python tests.py start k7m2x9p4qw8r feladat2` (ez az ablak a tesztek naplója), és egy másik ablakban a saját programja, majd egy harmadik ablakban: `python tests.py test feladat2 <port>`.

Fontos: a `tesztek` mappa obfuszkált kódját „fekete dobozként” kell kezelni, a rá alkalmazott kódvisszafejtés („reverse engineering”) csalásnak minősül.

3. feladat: azonosítási proxy (8 pont)

3. feladat

Készítsünk egy proxy programot, amely kétféle kliens kéréseit kezeli és továbbítja az Azonosítási Szerver felé! A megoldás helye: `feladat3/proxy.py`, indítása: `python proxy.py <port> <szerver_port>`. Az Azonosítási Szervert és a klienseket a tesztprogram futtatja, a szerver az UDP `20002`-es porton figyel: `python proxy.py 10002 20002`

- A Felhasználó Kliens bejelentkezési kérelmet küld.
- A Vendég Kliens információlekérési kérelmet küld.
- A Proxy az Azonosítási Szerver válaszát a megfelelő kliensnek küldi vissza.

A kliensek mindig TCP-n kapcsolódnak a Proxyhoz. Mindkét kliens esetén a Proxyhoz menő üzenet formátuma: kliens_típus, felhasználónév `(i 20s)`. A kliens_típus értékei: 0 = Felhasználó Kliens, 1 = Vendég Kliens.

A Proxy a `select` segítségével egyszerre több kliens kiszolgálására is képes. A Proxy az Azonosítási Szerver felé más formátumban továbbítja az üzeneteket:

- Felhasználó Kliens esetén: (`"LOGIN"`, felhasználónév), formátum: `(5s 20s)`
- Vendég Kliens esetén: (`"GUEST"`, felhasználónév), formátum: `(5s 20s)`

A Proxy az Azonosítási Szerverrel UDP-n keresztül kommunikál, és minden kliens kérését ezen keresztül továbbítja. A szerver bytesztring válasza változatlanul visszakerül a megfelelő klienshez.



Ellenőrzés. Töltse le és csomagolja ki a zárthelyi csomagját (zip), a feladatot a kicsomagolt mappában oldja meg! A Docker Desktopnak futnia kell. A csomag mappájában (ahol a `tests.py` van):

```
python tests.py check R8mK2vX7pQ4n feladat3
```

A következő kód (`R8mK2vX7pQ4n`) ennek a feladatnak a saját kódja: pontosan másolja ki (a kis- és nagybetű számít)! A `check` elindítja a programját, lefuttatja a teszteket, és a végén kiírja a kódokat: ezeket őrizze meg. Kézi mód fejlesztéshez: `python tests.py start R8mK2vX7pQ4n feladat3` (ez az ablak a tesztek naplója), és egy másik ablakban a saját programja, majd egy harmadik ablakban: `python tests.py test feladat3 <port>`.

Fontos: a `tesztek` mappa obfuszkált kódját „fekete dobozként” kell kezelni, a rá alkalmazott kódvisszafejtés („reverse engineering”) csalásnak minősül.